Лекция 6
Ядровые методы

Е. А. Соколов
ФКН ВШЭ

6 ноября 2026 г.

1 Ядровые методы

Нам уже известно некоторое количество способов преобразования признаков: можно добавлять признаки более высоких порядков, логарифмировать их или применять другие нелинейные преобразования, отбирать с помощью L_1-регуляризации или, например, можно порождать новые признаки с помощью решающих деревьев. Мы изучали и метод главных компонент, который позволяет формировать новые признаки как линейные комбинации исходных. В данной лекции мы обсудим ещё один подход к изменению признакового пространства: ядра, которые позволяют повышать размерность пространства без вычислительных трудностей.

§1.1 Восстановление нелинейных зависимостей линейными методами

Линейные методы классификации и регрессии являются хорошо изученными и обоснованными, однако предположение о линейной зависимости зачастую оказывается неверным в задачах машинного обучения. Оказывается, что линейные методы можно применять и для восстановления нелинейных зависимостей, если предварительно перейти к новым признакам.

Рассмотрим простой пример. На рис. 1 слева показана двумерная выборка с двумя классами, разделяющая поверхность для которой никак не может быть приближена гиперплоскостью. В то же время, если добавить третий признак $x_3 = x_1^2 + x_2^2$, то выборку можно будет идеально разделить гиперплоскостью вида $x_3 = C$ (рис. 1 справа). Такое пространство называется *спрямляющим*. В новом признаковом пространстве разделяющая поверхность является линейной, однако после ее проецирования на исходное пространство она окажется нелинейной.

Модель, в которой зависимость ищется как линейная комбинация нелинейных функций от выборки, называется *линейной моделью над базисными функциями* [1]. Например, для задачи регрессии она имеет вид

$$a(x) = \sum_{j=1}^{m} w_j \varphi_j(x),$$

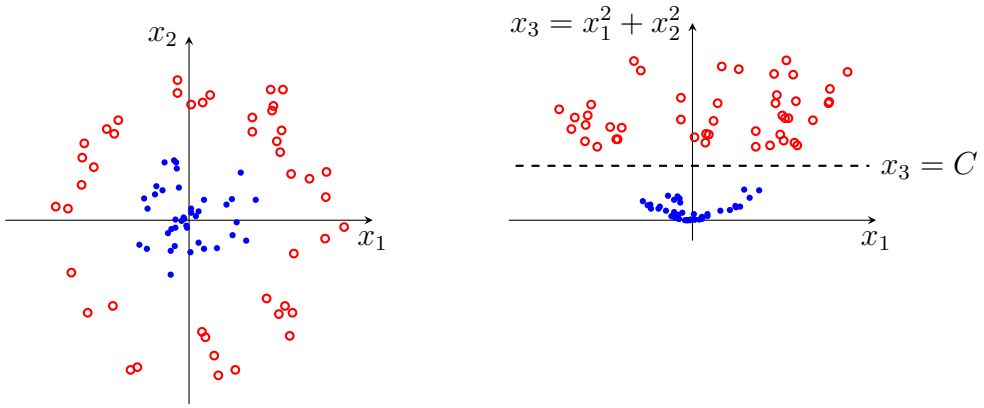


Рис. 1. Слева: выборка с нелинейной разделяющей поверхностью, классы обозначены разными цветами. Справа: та же выборка после добавления третьего признака; изображена проекция на первый и третий признаки. Ось x_3 сжата.

где $\varphi_j(x)$ — произвольные нелинейные функции от признаков (*базисные* функции). Проблема заключается в том, что на практике заранее нельзя сказать, какие именно базисные функции нужно взять, чтобы добиться линейной разделимости, поэтому приходится брать сразу большой набор таких функций (например, все мономы не больше определенной степени). В этом случае число признаков оказывается очень большим, из-за чего процесс обучения становится трудоемким как по времени, так и по памяти. Вторая проблема — переобучение: признаков становится больше, чем объектов, и без регуляризации ничего хорошего не выйдет. Однако в некоторых случаях оказывается, что достаточно уметь быстро вычислять скалярные произведения объектов друг на друга.

§1.2 Двойственное представление для линейной регрессии

Рассмотрим задачу построения линейной регрессии с квадратичной функцией потерь и квадратичным регуляризатором:

$$Q(w) = \frac{1}{2}\sum_{i=1}^{\ell}\left\{\sum_{j=1}^{m} w_j\varphi_j(x_i) - y_i\right\}^2 + \frac{\lambda}{2}\|w\|^2 = \frac{1}{2}\|\Phi w - y\|^2 + \frac{\lambda}{2}\|w\|^2 \to \min_w,$$

где Φ — матрица размера $\ell \times m$, в которой i-я строка представлена вектором $(\varphi_1(x_i), \ldots, \varphi_m(x_i))$. Дифференцируя функционал $Q(w)$ и приравнивая его нулю, получаем

$$\Phi^T(\Phi w - y) + \lambda w = 0, \qquad w = -\frac{1}{\lambda}\Phi^T(\Phi w - y).$$

Отсюда следует, что решение является линейной комбинацией строк матрицы Φ, то есть объектов $\varphi(x_i)$:

$$w = \Phi^T a,$$

где за a мы обозначили вектор $-\frac{1}{\lambda}(\Phi w - y)$. Подставим это представление в функционал:

$$Q(a) = \frac{1}{2}\|\Phi\Phi^T a - y\|^2 + \frac{\lambda}{2}a^T\Phi\Phi^T a \to \min_a.$$

Заметим, что теперь функционал зависит не от самой матрицы признаков Φ, а от ее произведения на саму себя $\Phi\Phi^T$. Это матрица скалярных произведений всех возможных пар объектов, называемая также *матрицей Грама*. Будем обозначать ее через

$$K = \Phi\Phi^T = \big(\langle\varphi(x_i), \varphi(x_j)\rangle\big)_{i,j=1}^{\ell} = \big(k(x_i, x_j)\big)_{i,j=1}^{\ell},$$

где $\varphi(x_i) = (\varphi_1(x_i), \ldots, \varphi_m(x_i))$, а $k(x_i, x_j)$ — скалярное произведение объектов, называемое также *функцией ядра*. Размер матрицы K — $\ell \times \ell$, и он не зависит от числа признаков m.

Можно показать, что оптимальный вектор a имеет вид

$$a = (K + \lambda I)^{-1} y.$$

Действительно, приравнивая градиент $Q(a)$ к нулю, получаем $K(Ka - y) + \lambda Ka = K\big((K+\lambda I)a - y\big) = 0$, и указанный вектор a обращает в ноль выражение в скобках. Функция регрессии при этом запишется как

$$y(x) = \langle w, \varphi(x)\rangle = w^T\varphi(x) = a^T\Phi\varphi(x) = k(x)^T(K + \lambda I)^{-1}y,$$

где $k(x) = (k(x, x_1), \ldots, k(x, x_\ell))$ — вектор скалярных произведений нового объекта x на объекты обучающей выборки.

Итак, нам удалось переписать функционал и модель так, что они зависят лишь от скалярных произведений объектов. В этом случае при росте размерности нового (спрямляющего) признакового пространства количество требуемой памяти остается константным и имеет порядок ℓ^2. Далее мы покажем, что и вычисление скалярного произведения можно организовать так, что оно будет зависеть лишь от размерности исходного признакового пространства.

Чем за это платим. Прямое представление требует $O(\ell m)$ памяти и решения системы размера $m \times m$, двойственное — $O(\ell^2)$ памяти и системы размера $\ell \times \ell$, то есть $O(\ell^3)$ операций. Предсказание для нового объекта в прямом представлении стоит $O(m)$, в двойственном — $O(\ell)$ вычислений ядра: сумма идёт по всей обучающей выборке. Поэтому ядровые методы выгодны, когда объектов меньше, чем признаков в спрямляющем пространстве, и перестают быть выгодными уже при сотнях тысяч объектов. Что с этим делать, обсудим на следующей лекции.

Теорема о представлении. То, что решение раскладывается по объектам обучающей выборки, не является особенностью квадратичной функции потерь. Рассмотрим произвольную задачу вида

$$\sum_{i=1}^{\ell} L\big(y_i, \langle w, \varphi(x_i)\rangle\big) + \frac{\lambda}{2}\|w\|^2 \to \min_w$$

с любой функцией потерь L. Разложим w на составляющую из линейной оболочки объектов $\varphi(x_1), \ldots, \varphi(x_\ell)$ и ортогональную к ней: $w = w_\parallel + w_\perp$. Скалярные произведения $\langle w, \varphi(x_i)\rangle$ от $w_\perp$ не зависят, а регуляризатор $\|w\|^2 = \|w_\parallel\|^2 + \|w_\perp\|^2$ при $w_\perp \neq 0$ только растёт. Значит, в точке минимума $w_\perp = 0$ и $w = \sum_{i=1}^{\ell} a_i\varphi(x_i)$. Это *теорема*

о представлении (representer theorem)[1]. Подставим разложение в функционал. Скалярное произведение с j-м объектом превращается в линейную комбинацию значений ядра:

$$\langle w, \varphi(x_j)\rangle = \Big\langle \sum_{i=1}^{\ell} a_i \varphi(x_i), \varphi(x_j)\Big\rangle = \sum_{i=1}^{\ell} a_i \langle \varphi(x_i), \varphi(x_j)\rangle = \sum_{i=1}^{\ell} a_i K(x_i, x_j) = (Ka)_j,$$

а регуляризатор — в квадратичную форму с матрицей Грама:

$$\|w\|^2 = \Big\langle \sum_{i=1}^{\ell} a_i \varphi(x_i), \sum_{j=1}^{\ell} a_j \varphi(x_j)\Big\rangle = \sum_{i,j=1}^{\ell} a_i a_j K(x_i, x_j) = a^T K a.$$

Получаем задачу относительно $a \in \mathbb{R}^\ell$, в которую объекты входят только через матрицу Грама:

$$\sum_{i=1}^{\ell} L\big(y_i, (Ka)_i\big) + \frac{\lambda}{2} a^T K a \to \min_a.$$

Так получаются ядровая логистическая регрессия, ядровой SVM и вообще ядровой вариант любой линейной модели с L_2-регуляризатором; отличие лишь в том, что для квадратичной потери ответ выписывается явно, а для остальных задачу приходится решать численно.

§1.3 SVM и kernel trick

Переход к новому признаковому пространству можно применять и в задачах классификации:

$$a(x) = \operatorname{sign}(\langle w, \varphi(x)\rangle + b).$$

В частности, к задаче метода опорных векторов можно построить двойственную (на следующей лекции мы сделаем это подробно):

$$\begin{cases} \sum\limits_{i=1}^{\ell} \lambda_i - \dfrac{1}{2} \sum\limits_{i,j=1}^{\ell} \lambda_i \lambda_j y_i y_j \langle x_i, x_j\rangle \to \max\limits_{\lambda} \\ 0 \leqslant \lambda_i \leqslant C, \quad i = 1, \ldots, \ell, \\ \sum\limits_{i=1}^{\ell} \lambda_i y_i = 0. \end{cases}$$

После того, как она решена, новые объекты классифицируются с помощью алгоритма

$$a(x) = \operatorname{sign}\left(\sum_{i=1}^{\ell} \lambda_i y_i \langle x_i, x\rangle + b\right).$$

[1] Schölkopf B., Herbrich R., Smola A. A Generalized Representer Theorem. COLT, 2001.

Заметим, что как оптимизационная задача, так и итоговый классификатор зависят лишь от скалярных произведений объектов. Подставляя вместо скалярного произведения функцию ядра, мы будем настраивать классификатор в произвольном признаковом пространстве. Такая подмена получила в англоязычной литературе название *kernel trick*. У SVM есть и дополнительное преимущество перед ядровой регрессией: большинство коэффициентов λ_i оказываются нулевыми, и в предсказании участвуют только опорные объекты.

§1.4 Ядра

Ядром мы будем называть функцию $K(x,z)$, представимую в виде скалярного произведения в некотором пространстве: $K(x,z) = \langle \varphi(x), \varphi(z) \rangle$, где $\varphi : \mathbb{X} \to H$ — отображение из исходного признакового пространства в некоторое *спрямляющее пространство*. Ядро содержит в себе много информации о спрямляющем пространстве и позволяет производить в нем различные операции, не зная самого отображения $\varphi(x)$. Например, норма и расстояние в спрямляющем пространстве выражаются через ядро:

$$\|\varphi(x)\| = \sqrt{K(x,x)}, \qquad \|\varphi(x) - \varphi(z)\|^2 = K(x,x) - 2K(x,z) + K(z,z).$$

Поэтому ядро можно подставить и в любой метрический метод — скажем, в kNN.

1.4.1 Построение ядер

Самый простой способ задать ядро — в явном виде построить отображение $\varphi(x)$ в спрямляющее признаковое пространство. Тогда ядро определяется как скалярное произведение в этом пространстве: $K(x,z) = \langle \varphi(x), \varphi(z) \rangle$. При таком способе, однако, возникают проблемы с ростом вычислительной сложности, о которых уже было сказано выше.

Допустим, в качестве новых признаков мы хотим взять всевозможные произведения исходных признаков. Определим соответствующее отображение

$$\varphi(x) = (x_i x_j)_{i,j=1}^{d} \in \mathbb{R}^{d^2}$$

и найдем ядро:

$$\begin{aligned} K(x,z) = \langle \varphi(x), \varphi(z) \rangle &= \langle (x_i x_j)_{i,j=1}^{d}, (z_i z_j)_{i,j=1}^{d} \rangle = \\ &= \sum_{i,j=1}^{d} x_i x_j z_i z_j = \\ &= \sum_{i=1}^{d} x_i z_i \sum_{j=1}^{d} x_j z_j = \\ &= \langle x, z \rangle^2. \end{aligned}$$

Таким образом, ядро выражается через скалярное произведение в исходном пространстве, и для его вычисления необходимо порядка d операций (в то время как прямое вычисление ядра потребовало бы $O(d^2)$ операций).

1.4.2 Неявное задание ядра

Пример с мономами показал, что можно определить ядро так, что оно не будет в явном виде использовать отображение объектов в новое признаковое пространство. Но как убедиться, что функция $K(x,z)$ определяет скалярное произведение в некотором пространстве? Скажем, является ли ядром функция $\sin\langle x,z\rangle + e^{\langle x,z\rangle}$? К этому вопросу мы вернёмся. Ответ на этот вопрос дает *теорема Мерсера*[2]: функция $K(x,z)$ является ядром тогда и только тогда, когда:

1. Она симметрична: $K(x,z) = K(z,x)$.
2. Она неотрицательно определена, то есть для любой конечной выборки $(x_1,\ldots,x_\ell)$ матрица $K = \big(K(x_i,x_j)\big)_{i,j=1}^{\ell}$ является неотрицательно определённой.

Необходимость проверяется в одну строку: $v^T K v = \sum_{i,j} v_i v_j \langle\varphi(x_i),\varphi(x_j)\rangle = \big\|\sum_i v_i\varphi(x_i)\big\|^2 \geqslant 0$. Достаточность — содержательная часть теоремы.

Проверять условия теоремы Мерсера, однако, может быть достаточно трудно. Поэтому для построения ядер, как правило, пользуются несколькими базовыми ядрами и операциями над ними, сохраняющими симметричность и неотрицательную определенность.

Теорема 1.1 ([2]). *Пусть $K_1(x,z)$ и $K_2(x,z)$ — ядра, заданные на множестве $\mathbb{X}$, $f(x)$ — вещественная функция на $\mathbb{X}$, $\varphi : \mathbb{X} \to \mathbb{R}^N$ — векторная функция на $\mathbb{X}$, K_3 — ядро, заданное на $\mathbb{R}^N$. Тогда следующие функции являются ядрами:*

1. $K(x,z) = K_1(x,z) + K_2(x,z)$,
2. $K(x,z) = \alpha K_1(x,z)$, $\alpha > 0$,
3. $K(x,z) = K_1(x,z)K_2(x,z)$,
4. $K(x,z) = f(x)f(z)$,
5. $K(x,z) = K_3(\varphi(x),\varphi(z))$.

Теорему можно доказать конструктивно: для каждого правила выпишем спрямляющее пространство. Пусть ядру K_1 соответствует отображение $\varphi_1 : \mathbb{X} \to \mathbb{R}^{d_1}$, а ядру K_2 — отображение $\varphi_2 : \mathbb{X} \to \mathbb{R}^{d_2}$. Для простоты считаем спрямляющие пространства конечномерными; о том, что меняется в общем случае, скажем после.

1. Сумме ядер соответствует конкатенация признаков: $\varphi(x) = (\varphi_1(x),\varphi_2(x)) \in \mathbb{R}^{d_1+d_2}$.
2. Умножению на α — масштабирование: $\varphi(x) = \sqrt{\alpha}\,\varphi_1(x)$.

[2] Строго говоря, этот критерий — характеризация положительно определённых функций (теорема Мура–Аронсайна). Теорема Мерсера — более специальное утверждение о непрерывных ядрах на компакте и их разложении по собственным функциям. В курсах по машинному обучению критерий традиционно называют именем Мерсера.

3. Произведению ядер — все попарные произведения признаков из двух пространств:

$$\varphi(x) = \big((\varphi_1(x))_i(\varphi_2(x))_j\big)_{i,j=1}^{d_1,d_2} \in \mathbb{R}^{d_1 d_2}.$$

Действительно,

$$\langle \varphi(x), \varphi(z)\rangle = \sum_i (\varphi_1(x))_i(\varphi_1(z))_i \sum_j (\varphi_2(x))_j(\varphi_2(z))_j = K_1(x,z)K_2(x,z).$$

Пример с $\langle x, z\rangle^2$ выше — частный случай при $\varphi_1 = \varphi_2 = \mathrm{id}$.

4. Ядру $f(x)f(z)$ — один-единственный признак $\varphi(x) = f(x)$.

5. Композиции — композиция отображений: $\varphi(x) = \varphi_3(\varphi(x))$, где φ_3 задаёт ядро K_3.

Это полезно помнить: всякий раз, когда ядро собирается из базовых по этим правилам, его спрямляющее пространство можно выписать явно.

Если спрямляющие пространства бесконечномерны (а такие ядра нам скоро встретятся), конструкции остаются теми же, но требуют аккуратности: сумме ядер отвечает прямая сумма $H_1 \oplus H_2$, а произведению — тензорное произведение $H_1 \otimes H_2$, в котором $\langle a \otimes b, c \otimes d\rangle = \langle a, c\rangle\langle b, d\rangle$. Для пространств со счётным базисом выкладка для произведения проходит дословно: двойной ряд сходится абсолютно, так как $\sum_i |(\varphi_1(x))_i(\varphi_1(z))_i| \leqslant \|\varphi_1(x)\|\|\varphi_1(z)\|$ по неравенству Коши–Буняковского, и суммы можно переставлять.

Есть и доказательство, которому размерность безразлична — через критерий. Симметричность во всех пяти случаях очевидна. Матрица Грама суммы ядер — сумма двух неотрицательно определённых матриц, она неотрицательно определена; умножение на $\alpha > 0$ ничего не портит. Матрица Грама произведения ядер — покомпонентное (адамарово) произведение двух неотрицательно определённых матриц, и оно неотрицательно определено по теореме Шура. Для $f(x)f(z)$ матрица имеет вид vv^T при $v = (f(x_1), \ldots, f(x_\ell))$, а для композиции — это матрица Грама ядра K_3 на выборке $\varphi(x_1), \ldots, \varphi(x_\ell)$. Конструктивные отображения выше при этом остаются полезной иллюстрацией.

Теорема 1.2 ([3]). *Пусть $K_1(x,z), K_2(x,z), \ldots$ — последовательность ядер, причем предел*

$$K(x,z) = \lim_{n\to\infty} K_n(x,z)$$

существует для всех x и z. Тогда $K(x,z)$ — ядро.

Рассмотрим некоторые примеры построения ядер.

1.4.3 Полиномиальные ядра

Пусть $p(v)$ — многочлен с положительными коэффициентами. Тогда $K(x,z) = p(\langle x, z\rangle)$ — ядро: каждая степень $\langle x, z\rangle^i$ — произведение ядер, положительный

коэффициент — правило 2, свободный член $a_0 = \sqrt{a_0}\sqrt{a_0}$ — правило 4, сумма — правило 1. Также известно, что $p(K(x,z))$ — ядро для любого ядра $K(x,z)$.

Рассмотрим частный случай полиномиального ядра:

$$K_m(x,z) = (\langle x,z\rangle + R)^m .$$

Распишем степень, воспользовавшись формулой бинома Ньютона:

$$K_m(x,z) = \sum_{i=0}^{m} C_m^i R^{m-i} \langle x,z\rangle^i.$$

Поскольку коэффициенты при скалярных произведениях $C_m^i R^{m-i}$ положительны, то данное ядро действительно является ядром. Если расписать скалярные произведения, то можно убедиться, что оно соответствует переводу набора признаков во всевозможные мономы над признаками степени не больше m, причем моном степени i имеет вес $\sqrt{C_m^i R^{m-i}}$ (с точностью до мультиномиальных коэффициентов, которые появляются при раскрытии $\langle x,z\rangle^i$: например, в $\langle x,z\rangle^2$ моном x_1x_2 входит с весом $\sqrt{2}$, а x_1^2 — с весом 1).

Заметим, что параметр R контролирует относительный вес при мономах больших степеней. Например, отношение веса при мономе степени $m-1$ к весу при мономе первой степени равно

$$\sqrt{\frac{C_m^{m-1}R}{C_m^1 R^{m-1}}} = \sqrt{\frac{1}{R^{m-2}}},$$

то есть по мере увеличения R вес при мономах старших степеней будет становиться очень небольшим по сравнению с весом при остальных мономах. Можно сказать, что параметр R контролирует сложность модели.

1.4.4 Гауссовские ядра

Гауссовское ядро определяется как

$$K(x,z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right).$$

Покажем, что оно действительно является ядром.

Покажем сначала, что функция $\exp(\langle x,z\rangle)$ является ядром. Представим ее в виде предела последовательности:

$$\exp(\langle x,z\rangle) = \sum_{k=0}^{\infty} \frac{\langle x,z\rangle^k}{k!} = \lim_{n\to\infty} \sum_{k=0}^{n} \frac{\langle x,z\rangle^k}{k!}.$$

Каждый член предельной последовательности является многочленом с положительными коэффициентами, и поэтому является ядром. Предел существует во всех точках (x,z), поскольку ряд Тейлора для функции e^x сходится на всей числовой прямой. Значит, по теореме 1.2 данная функция является ядром.

Аналогично можно доказать, что функция $\exp(\langle x,z\rangle/\sigma^2)$ также является ядром. Гауссовское ядро легко получить из данного путём замены преобразования $\varphi(x)$

на $\varphi(x)/\|\varphi(x)\|$: нормировка $K(x,z)/\sqrt{K(x,x)K(z,z)}$ даёт $\exp\big(\langle x,z\rangle/\sigma^2 - \|x\|^2/2\sigma^2 - \|z\|^2/2\sigma^2\big)$, а это и есть $\exp(-\|x-z\|^2/2\sigma^2)$. Такую нормировку можно применить к любому ядру; результат называют косинусным ядром, поскольку это косинус угла между $\varphi(x)$ и $\varphi(z)$, а спрямляющее пространство — проекция исходного на единичную сферу.

Заметим, что можно построить гауссово ядро, используя любое другое ядро $K(x,z)$. В этом случае оно примет вид

$$\exp\left(-\frac{\|\varphi(x)-\varphi(z)\|^2}{2\sigma^2}\right) = \exp\left(-\frac{K(x,x)-2K(x,z)+K(z,z)}{2\sigma^2}\right).$$

Здесь мы расписали расстояние между векторами $\|\varphi(x)-\varphi(z)\|^2$ в спрямляющем пространстве через функцию ядра.

Спрямляющее пространство. Какому спрямляющему пространству соответствует гауссовское ядро? Оно является пределом последовательности полиномиальных ядер при стремлении степени ядра к бесконечности, что наталкивает на мысль, что и спрямляющее пространство будет бесконечномерным. Чтобы показать это формально, нам понадобится следующее утверждение.

Утв. 1.3 ([4]). *Пусть $x_1,\dots,x_\ell$ — различные точки пространства $\mathbb{R}^d$. Тогда матрица*

$$G = \left[\exp\left(-\frac{\|x_i-x_j\|^2}{2\sigma^2}\right)\right]_{i,j=1}^{\ell}$$

является невырожденной при $\sigma > 0$.

Вспомним также факт из линейной алгебры: матрица Грама системы точек $x_1,\dots,x_\ell$ невырождена тогда и только тогда, когда эти точки линейно независимы. Поскольку матрица из утверждения 1.3 является матрицей Грама для точек $x_1,\dots,x_\ell$ в спрямляющем пространстве гауссова ядра, то заключаем, что в данном пространстве существует сколь угодно много линейно независимых точек. Значит, данное пространство является бесконечномерным.

У этого утверждения есть практическое следствие (задача с семинара). Возьмём любую выборку $(x_i,y_i)_{i=1}^{\ell}$ с различными объектами и любыми метками $y_i \in \{-1,+1\}$ и будем искать $w = \sum_j a_j\varphi(x_j)$ такой, что $\langle w,\varphi(x_i)\rangle = y_i$ для всех i. Это система $Ga = y$ с невырожденной матрицей, и у неё всегда есть решение. Значит, в спрямляющем пространстве гауссова ядра *любая* выборка линейно разделима, какими бы ни были метки, в том числе случайными. Это не повод для радости: модель, способная идеально выучить любой шум, без регуляризации переобучится. Поэтому в ядровом SVM параметр C и ширина σ подбираются вместе, и оба влияют на сложность.

Можно показать это и менее формально. Распишем функцию ядра:

$$K(x,z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right) = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right)\exp\left(-\frac{\|z\|^2}{2\sigma^2}\right)\exp\left(\frac{\langle x,z\rangle}{\sigma^2}\right) =$$
$$= \{\text{раскладываем экспоненту в ряд}\} =$$
$$= \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right)\exp\left(-\frac{\|z\|^2}{2\sigma^2}\right)\sum_{k=0}^{\infty}\frac{\langle x,z\rangle^k}{k!\sigma^{2k}}.$$

Легко видеть, что для получения k-го слагаемого в спрямляющем пространстве должны быть все мономы степени k над исходными признаками. Поскольку всего в сумме бесконечное число слагаемых, то и размерность спрямляющего пространства должна быть бесконечной.

На гауссово ядро полезно смотреть и с третьей стороны, без мономов. Положим $\varphi(x)$ равным функции $z \mapsto K(x,z)$, то есть «колоколу» с центром в точке x. Скалярное произведение двух таких функций определим так, чтобы $\langle \varphi(x), \varphi(z) \rangle = K(x,z)$; можно проверить, что это корректное скалярное произведение на линейных комбинациях колоколов. Тогда ядровая модель $a(x) = \sum_i a_i K(x_i, x)$ — это просто взвешенная сумма колоколов, расставленных в объектах обучающей выборки, а обучение — подбор высот. Такое построение работает для любого ядра и называется каноническим спрямляющим пространством (RKHS, reproducing kernel Hilbert space): если явное отображение неизвестно, всегда можно считать, что $\varphi(x) = K(x, \cdot)$.

Роль параметра σ. Заметим, что параметр σ в разложении гауссова ядра в ряд Тейлора входит в коэффициент перед слагаемым $\langle x, z \rangle^k$ как $1/\sigma^{2k}$. Его роль аналогична параметру R в полиномиальных ядрах. Маленькие значения σ соответствуют большим значениям R: чем меньше σ, тем больше вес при мономах большой степени, тем больше риск переобучения.

Предельные случаи делают это наглядным. При $\sigma \to 0$ ядро $K(x_i, x_j)$ стремится к нулю для любых двух различных объектов, матрица Грама — к единичной, все объекты становятся опорными, и классификатор $\sum_i \lambda_i y_i K(x_i, x)$ в каждой точке определяется ближайшим объектом выборки: получается метод одного ближайшего соседа, который идеально запоминает обучение. При $\sigma \to \infty$ в разложении остаются только слагаемые низких степеней, и модель становится почти линейной. Промежуточные σ интерполируют между этими крайностями; на следующей лекции мы увидим, что SVM с гауссовым ядром вообще можно понимать как взвешенный метод ближайших соседей.

§1.5 Больше примеров ядер

Полиномиальные и гауссовы ядра — стандартный набор для числовых признаков. Но ядро — это просто мера сходства, удовлетворяющая теореме Мерсера, и его можно задать на объектах любой природы. Для каждого примера ниже выпишем спрямляющее пространство.

Ядра на множествах. Пусть объекты — подмножества конечного множества D (слова в тексте, товары в корзине, теги у видео). Простейшее ядро — число общих элементов:

$$K(A,B) = |A \cap B|, \qquad \varphi(A) = \big([u \in A]\big)_{u \in D}.$$

Спрямляющее пространство — индикаторы элементов, то есть обычный мешок слов. Можно учитывать не только отдельные элементы, но и все их комбинации (задача с семинара):

$$K(A,B) = 2^{|A \cap B|}, \qquad \varphi(A) = \big([U \subseteq A]\big)_{U \subseteq D}.$$

Действительно, $\langle\varphi(A),\varphi(B)\rangle$ — число подмножеств U, лежащих и в A, и в B, то есть число подмножеств $A\cap B$. Спрямляющее пространство имеет размерность $2^{|D|}$, а ядро считается за $O(|D|)$.

Ядра на гистограммах. *Гистограмма* — это вектор $x\in\mathbb{R}^d_+$ с неотрицательными координатами, где x_j — сколько раз в объекте встретился j-й элемент словаря (или доля таких вхождений, тогда $\sum_j x_j=1$). Мешок слов из предыдущего абзаца — гистограмма над словарём слов; гистограмма яркостей пикселей — над отрезками шкалы яркости. Для гистограмм естественная мера сходства — насколько они перекрываются по каждой координате, и это *ядро пересечения*:

$$K(x,z)=\sum_{j=1}^{d}\min(x_j,z_j).$$

Если x и z — индикаторы множеств, это снова $|A\cap B|$; для счётчиков — число общих вхождений.

Выпишем спрямляющее пространство явно. Заметим, что $\min(x_j,z_j)$ — длина пересечения отрезков $[0,x_j]$ и $[0,z_j]$, то есть

$$\min(x_j,z_j)=\int_0^\infty [t\leqslant x_j][t\leqslant z_j]\,dt.$$

Значит, j-й координате отвечает признак-функция $\varphi_j(x):t\mapsto[t\leqslant x_j]$ — ступенька, равная единице на $[0,x_j]$ и нулю правее. Спрямляющее пространство — d копий пространства функций на $[0,\infty)$ со скалярным произведением

$$\langle f,g\rangle=\sum_{j=1}^{d}\int_0^\infty f_j(t)g_j(t)\,dt,$$

а $\varphi(x)=(\varphi_1(x),\ldots,\varphi_d(x))$. Тогда $\langle\varphi(x),\varphi(z)\rangle=\sum_j\int[t\leqslant x_j][t\leqslant z_j]\,dt=$ $=\sum_j\min(x_j,z_j)=K(x,z)$. Пространство бесконечномерное, но по сути это то же $|A\cap B|$: каждая координата заменена «множеством» $[0,x_j]$.

Если значения x_j — целые от 0 до m (счётчики), ступеньки можно заменить конечным вектором: $\varphi_j(x)=\big([1\leqslant x_j],[2\leqslant x_j],\ldots,[m\leqslant x_j]\big)$ — унарный код числа x_j. Тогда $\langle\varphi_j(x),\varphi_j(z)\rangle=\#\{k\colon k\leqslant x_j,\ k\leqslant z_j\}=\min(x_j,z_j)$, и спрямляющее пространство — $\mathbb{R}^{dm}$: ядро пересечения — это обычное скалярное произведение унарных кодов.

Главное применение этого ядра — классификация изображений до появления свёрточных сетей. Тогда изображение описывали так же, как текст: в нём находили локальные фрагменты (например, окрестности углов и пятен), каждый описывали вектором из $\sim$100 чисел, все такие векторы по большой коллекции изображений кластеризовали методом k средних на несколько тысяч групп, и центр каждой группы объявляли «визуальным словом» — по аналогии со словами в мешке слов. Изображение превращалось в гистограмму: сколько его фрагментов попало в каждый кластер. Классификация изображений сводилась к классификации гистограмм, и SVM с ядром пересечения был для неё стандартным выбором[3].

[3]Lazebnik S., Schmid C., Ponce J. Beyond Bags of Features: Spatial Pyramid Matching for Recognizing Natural Scene Categories. CVPR, 2006.

Все мономы без повторений. Ядро $K(x,z) = \prod_{j=1}^{d}(1 + x_j z_j)$ (задача с семинара) при раскрытии скобок даёт сумму $\sum_{S\subseteq\{1,\dots,d\}} \prod_{j\in S} x_j z_j$, то есть спрямляющее пространство состоит из всех мономов, в которых каждый признак входит не более одного раза:

$$\varphi(x) = \Big(\prod_{j\in S} x_j\Big)_{S\subseteq\{1,\dots,d\}}.$$

Размерность 2^d, стоимость вычисления ядра $O(d)$.

Ядра на строках. Для строк s, t над алфавитом Σ естественное спрямляющее пространство — счётчики подстрок. *Спектральное* ядро использует все подстроки фиксированной длины p:

$$\varphi(s) = \big(\text{число вхождений } u \text{ в } s\big)_{u\in\Sigma^p}, \qquad K(s,t) = \langle\varphi(s), \varphi(t)\rangle,$$

то есть это мешок p-грамм. Ядро *всех подпоследовательностей* считает общие подпоследовательности, не обязательно непрерывные; спрямляющее пространство индексируется всеми строками $u \in \Sigma^*$, и явно его не построить, но само ядро вычисляется динамическим программированием за $O(|s| \cdot |t|)$. Именно такие примеры показывают смысл ядер: объекты не из $\mathbb{R}^d$, спрямляющее пространство необозримо, а ядро считается быстро.

Ядра на графах. Пусть объекты — графы с метками на вершинах: для молекулы вершины — атомы, метки — их типы. Метки должны быть дискретными: ниже они сравниваются только на равенство, поэтому числовые атрибуты вершин предварительно квантуют. Если меток нет, в качестве метки берут степень вершины (или просто одну и ту же метку для всех вершин — после первого шага перекраски она сама превратится в степень). Самый простой признак графа — сколько вершин каждой метки в нём есть: $\varphi(G) = (\text{число вершин с меткой } c)_c$. Скалярное произведение таких счётчиков — ядро, но очень грубое: оно не видит рёбер. Ядро Вейсфейлера–Лемана[4] исправляет это перекраской вершин. До появления графовых нейронных сетей оно было стандартным инструментом для графов.

Шаг 0. Каждая вершина v получает исходную метку $\ell_0(v)$.

Шаг $i = 1, \dots, h$. Для каждой вершины составляется строка: её текущая метка, а за ней — отсортированный список меток соседей. Например, вершина с меткой A и соседями с метками B и A даёт строку «A; A, B». Каждой новой строке присваивается новая метка из общего словаря (одна и та же строка в любом графе получает одну и ту же метку), и эта метка становится $\ell_i(v)$. После i шагов метка вершины кодирует структуру её окрестности радиуса i.

Признаки. После h шагов для каждого шага i и каждой метки c считаем, сколько вершин графа имеют метку c на шаге i. Спрямляющее пространство — все такие счётчики:

$$\varphi(G) = \big(n_{i,c}(G)\big)_{i=0,\dots,h;\ c\in\text{словарь}}, \qquad n_{i,c}(G) = \text{число вершин } G \text{ с меткой } c \text{ на шаге } i,$$

[4] Shervashidze N. et al. Weisfeiler-Lehman Graph Kernels. JMLR, 2011.

а ядро — их скалярное произведение: $K(G, G') = \sum_i \sum_c n_{i,c}(G)\, n_{i,c}(G')$. Размерность пространства — размер словаря, он растёт с h, но вычислять ядро по-прежнему дёшево: все шаги занимают $O(h \cdot |E|)$ на граф. Словарь меток при этом общий для всей базы графов, а не для пары: строка «A; A, B» в любом графе должна получать одну и ту же метку, иначе счётчики никогда не совпадут. Поэтому перекраску делают сразу для всех графов, с одной таблицей.

Вектор $\varphi(G)$ — гистограмма меток, и на неё можно поставить любое ядро для гистограмм, по правилу композиции из теоремы 1.1: например, ядро пересечения $\sum_{i,c} \min(n_{i,c}(G), n_{i,c}(G'))$, которое считает общие поддеревья с учётом кратности. Скалярное произведение выбрано в исходной статье по традиции ядер на структурных объектах: оно считает число *пар* совпадающих поддеревьев (метка, встретившаяся дважды в G и трижды в G', даёт вклад 6, а не 2) и удобно тем, что матрица Грама для всей базы — одно произведение разреженной матрицы счётчиков на транспонированную. Платят за это чувствительностью к размеру графов и к повторам, поэтому на практике ядро обычно нормируют косинусом.

Пример. G_1 — цепочка A–B–A, G_2 — цепочка A–A–B. На шаге 0 у обоих по две вершины A и одной B, счётчики совпадают. На шаге 1 в G_1 две вершины получают строку «A; B» и одна — «B; A, A», а в G_2 все три строки разные: «A; A», «A; A, B», «B; A». Общих меток на шаге 1 нет, поэтому $K(G_1, G_2) = 2 \cdot 2 + 1 \cdot 1 = 5$, тогда как $K(G_1, G_1) = 5 + 2^2 + 1^2 = 10$: ядро различило графы с одинаковым набором атомов, но разной структурой.

Связь с графовыми сетями. Сама перекраска — это тест Вейсфейлера–Лемана на изоморфизм: если после h шагов счётчики двух графов совпадают, тест считает графы неразличимыми. Он не всесилен: два треугольника и шестиугольник он не различит, потому что все вершины в них имеют степень 2 и одинаковые окрестности на любом шаге. Графовые нейронные сети обновляют представление вершины по представлениям соседей, то есть делают ту же перекраску, только с обучаемой функцией вместо словаря, и поэтому различают графы не лучше этого теста. Так что ограничения ядра и ограничения сетей — одни и те же, и этот пример ещё пригодится.

Ядро из обученной модели. Любая модель, которая строит представления объектов, задаёт ядро $K(x, z) = \langle \varphi(x), \varphi(z) \rangle$, где $\varphi(x)$ — выход сети. В этом смысле двухбашенная модель из лекции про рекомендации — обучаемое ядро между пользователем и товаром, а LFM — такое же ядро с явно заданными, а не вычисляемыми векторами. Разница с классическими ядровыми методами в том, откуда берётся φ: из формулы или из данных.

Что ядром не является. Ядро обязано быть неотрицательным на диагонали: $K(x, x) = \|\varphi(x)\|^2 \geqslant 0$. Уже это отсекает многие кандидаты. Скажем, $\sin\langle x, z \rangle$ — не ядро: при $d = 1$ и $x^2 = 3\pi/2$ получаем $K(x, x) = \sin(3\pi/2) = -1$. А вот функция $\sin\langle x, z \rangle + e^{\langle x, z \rangle}$, как ни странно, ядро: складывая ряды Тейлора, получаем $\sum_k c_k \langle x, z \rangle^k$, где $c_k = 2/k!$ при $k \equiv 1 \pmod 4$, $c_k = 0$ при $k \equiv 3 \pmod 4$ и $c_k = 1/k!$ при чётных k. Все коэффициенты неотрицательны, и по теореме 1.2 это ядро. Так что сумма ядра и не-ядра может оказаться ядром, и проверять нужно именно итоговую функцию. Другой известный пример — сигмоидное «ядро» $\tanh(\alpha \langle x, z \rangle + \beta)$,

которое предлагалось по аналогии с нейронными сетями и входило в старые библиотеки SVM. Оно не является ядром при большинстве значений параметров, хотя SVM с ним нередко работал: оптимизация просто решала невыпуклую задачу и находила какое-то локальное решение.

§1.6 Где возникают ядровые методы

Ядровой SVM уступил место бустингу на табличных данных и нейронным сетям на всём остальном, но сами ядра возникают в самых разных местах. Разберём четыре примера.

Гауссовские процессы. Ядровую регрессию можно получить из вероятностной модели. Пусть $y = f(x) + \varepsilon$, где $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ — шум измерения (повторный запуск в той же точке x даёт другой y), а f — случайная функция, априорное распределение которой — *гауссовский процесс*: для любого набора точек вектор $(f(x_1), \dots, f(x_\ell))$ нормален с нулевым средним и ковариационной матрицей $K = (K(x_i, x_j))_{i,j=1}^{\ell}$:

$$p\big(f(x_1), \dots, f(x_\ell)\big) = \mathcal{N}\big(0, K\big).$$

Это то же самое, что линейная модель $f(x) = \langle w, \varphi(x)\rangle$ с $w \sim \mathcal{N}(0, I)$: ковариация $f(x)$ и $f(z)$ равна $\langle \varphi(x), \varphi(z)\rangle = K(x, z)$. Добавим к обучающим точкам новую точку x и перейдём от f к наблюдениям $y = (y_1, \dots, y_\ell)$: шум независим и добавляет σ^2 на диагональ, так что совместное распределение

$$p\big(y, f(x)\big) = \mathcal{N}\left(0, \begin{pmatrix} K + \sigma^2 I & k(x) \\ k(x)^T & K(x,x) \end{pmatrix}\right), \qquad k(x) = \big(K(x, x_1), \dots, K(x, x_\ell)\big).$$

Условное распределение одной компоненты нормального вектора при известных остальных снова нормально, и по стандартной формуле для него

$$\begin{aligned} p\big(f(x) \,|\, y\big) &= \mathcal{N}\big(m(x), s^2(x)\big), \\ m(x) &= k(x)^T (K + \sigma^2 I)^{-1} y, \\ s^2(x) &= K(x,x) - k(x)^T (K + \sigma^2 I)^{-1} k(x). \end{aligned}$$

Среднее — в точности формула ядровой регрессии при $\lambda = \sigma^2$: регуляризатор оказывается дисперсией шума. Случайность f и случайность ε — разные вещи: первая описывает наше незнание функции до наблюдений, вторая — шум в самих наблюдениях. Без ε процесс после наблюдений проходил бы ровно через обучающие точки с нулевой дисперсией в них, а среднее превращалось бы в ядровую регрессию с $\lambda = 0$. Новое здесь — дисперсия: при $\sigma = 0$ она равна нулю в обучающих точках и растёт по мере удаления от них. Поэтому гауссовские процессы используют там, где каждое измерение дорого и нужно знать, где модель не уверена: активное обучение, суррогатные модели в физике и химии, байесовская оптимизация.

Байесовская оптимизация на гауссовских процессах — это подбор гиперпараметров по десяткам или сотням запусков: по среднему и дисперсии выбирается следующая точка, например, минимум нижней доверительной границы $m(x) - \kappa\, s(x)$ при минимизации качества (или максимум верхней при максимизации): κ задаёт баланс

между эксплуатацией и разведкой, большой κ тянет в области, где модель не уверена. Другой распространённый критерий — ожидаемое улучшение $\mathbb{E}\max(0, y_{\text{best}} - f(x))$ по апостериорному распределению $f(x)$, модель перестраивается, и так далее. В популярных библиотеках для подбора гиперпараметров и в AutoML-системах гауссовских процессов, как правило, нет: там используют более простые суррогатные модели, случайный поиск с ранней остановкой неудачных запусков или вообще обходятся без подбора, обучая несколько заранее выбранных конфигураций и собирая из них ансамбль. Причины понятны: гауссовский процесс стоит $O(\ell^3)$ по числу прошлых запусков, плохо переносит размерность больше десятка-двух и неудобен для категориальных и условных параметров, а в глубинном обучении гиперпараметры больших моделей и вовсе настраивают на маленьких и переносят по законам масштабирования.

Это не значит, что гауссовские процессы проигрывают по качеству. На соревновании по чёрно-ящичной оптимизации NeurIPS 2020, где на каждую задачу давалось 128 запусков, почти все лучшие решения были построены на гауссовских процессах, а случайному поиску для того же результата понадобилось бы примерно в сто раз больше запусков[5]. На малоразмерных непрерывных задачах гауссовские процессы обычно точнее других суррогатных моделей, а на многомерных и с условными параметрами уступают им[6]. Проигрыш в скорости окупается, когда запусков мало, а каждый дорог: именно поэтому гауссовские процессы остаются в сервисах вроде Google Vizier, где по умолчанию стоит GP-бандит, и в задачах с действительно дорогими экспериментами — подбор материалов, химия, настройка физических установок. Когда же один запуск стоит секунды, выгоднее сделать на порядок больше запусков дешёвым методом, чем тратить время на суррогатную модель. К подбору гиперпараметров мы ещё вернёмся в лекции про AutoML.

Сравнение распределений. Пусть K — заранее выбранное ядро, например гауссово, а φ — его спрямляющее отображение. Для распределения P на $\mathbb{X}$ определим его *вложение* $\mu_P = \mathbb{E}_{x\sim P}\varphi(x)$ — точку в спрямляющем пространстве. Здесь φ не зависит ни от P, ни от выборки, а P может быть любым распределением, для которого $\mathbb{E}\sqrt{K(x,x)} < \infty$; для ограниченных ядер, в том числе гауссова, это любое распределение. Математическое ожидание понимается в спрямляющем пространстве: μ_P — такой его элемент, что $\langle \mu_P, f\rangle = \mathbb{E}_{x\sim P} f(x)$ для всех f из этого пространства. Иными словами, μ_P хранит математические ожидания сразу всех функций из спрямляющего пространства. На практике P задано выборкой $x_1, \ldots, x_n$, и μ_P заменяется на среднее $\frac{1}{n}\sum_i \varphi(x_i)$.

Расстояние между распределениями определим как максимальное расхождение их средних по функциям из спрямляющего пространства с единичной нормой:

$$\operatorname{MMD}(P,Q) = \sup_{\|f\|\leqslant 1} \big(\mathbb{E}_{x\sim P} f(x) - \mathbb{E}_{y\sim Q} f(y)\big).$$

Отсюда название — maximum mean discrepancy. Поскольку $\mathbb{E}_P f(x) - \mathbb{E}_Q f(y) = \langle f, \mu_P - \mu_Q\rangle$, а супремум скалярного произведения по единичному шару равен норме,

[5] Turner R. et al. Bayesian Optimization is Superior to Random Search for Machine Learning Hyperparameter Tuning: Analysis of the Black-Box Optimization Challenge 2020. PMLR, 2021.

[6] Eggensperger K. et al. Towards an Empirical Foundation for Assessing Bayesian Optimization of Hyperparameters. NIPS Workshop on Bayesian Optimization, 2013.

$\mathrm{MMD}(P,Q) = \|\mu_P - \mu_Q\|$. Раскроем квадрат нормы и воспользуемся тем, что математическое ожидание выносится из скалярного произведения: при независимых $x \sim P$ и $y \sim Q$

$$\langle \mu_P, \mu_Q \rangle = \big\langle \mathbb{E}_x \varphi(x), \mathbb{E}_y \varphi(y) \big\rangle = \mathbb{E}_x \mathbb{E}_y \langle \varphi(x), \varphi(y) \rangle = \mathbb{E} K(x,y),$$

и аналогично $\langle \mu_P, \mu_P \rangle = \mathbb{E} K(x,x')$ для двух *независимых* копий $x, x' \sim P$ (если взять одну и ту же x, получится $\mathbb{E} K(x,x) = \mathbb{E}\|\varphi(x)\|^2$, а это другая величина). Итого

$$\mathrm{MMD}^2(P,Q) = \|\mu_P - \mu_Q\|^2 = \mathbb{E} K(x,x') + \mathbb{E} K(y,y') - 2\mathbb{E} K(x,y),$$

где $x, x' \sim P$ и $y, y' \sim Q$ независимы. Это та же формула, что и для расстояния между двумя объектами $\|\varphi(x) - \varphi(z)\|^2 = K(x,x) - 2K(x,z) + K(z,z)$, только вместо точек — средние по распределениям. По выборкам всё считается через одни только ядра, без φ.

Как именно: есть выборки $x_1, \ldots, x_n \sim P$ и $y_1, \ldots, y_m \sim Q$, и каждое математическое ожидание заменяется средним по парам. Если взять все пары, включая $i = j$, получится

$$\widehat{\mathrm{MMD}}_b^2 = \frac{1}{n^2} \sum_{i,j=1}^{n} K(x_i, x_j) + \frac{1}{m^2} \sum_{i,j=1}^{m} K(y_i, y_j) - \frac{2}{nm} \sum_{i=1}^{n} \sum_{j=1}^{m} K(x_i, y_j) =$$
$$= \Big\| \frac{1}{n} \sum_{i=1}^{n} \varphi(x_i) - \frac{1}{m} \sum_{j=1}^{m} \varphi(y_j) \Big\|^2$$

— норма разности выборочных средних в спрямляющем пространстве; эта оценка смещена, поскольку слагаемые $K(x_i, x_i)$ отвечают не независимым копиям, а одному и тому же объекту. Если пары $i = j$ внутри каждой выборки исключить, оценка становится несмещённой:

$$\widehat{\mathrm{MMD}}_u^2 = \frac{1}{n(n-1)} \sum_{i \neq j} K(x_i, x_j) + \frac{1}{m(m-1)} \sum_{i \neq j} K(y_i, y_j) - \frac{2}{nm} \sum_{i=1}^{n} \sum_{j=1}^{m} K(x_i, y_j).$$

Именно её имеют в виду, когда говорят о несмещённости ниже. Стоимость — матрица Грама объединённой выборки, $O((n+m)^2)$ вычислений ядра.

Ядро называется *характеристическим*, если разным распределениям соответствуют разные вложения: из $\mu_P = \mu_Q$ следует $P = Q$. Тогда $\mathrm{MMD} = 0$ только при $P = Q$, то есть MMD — расстояние между распределениями. Линейное ядро не характеристическое: $\mu_P = \mathbb{E}x$, и распределения с равными средними неразличимы; полиномиальное степени m помнит только моменты до m-го. Гауссово и лапласово ядра характеристические: их вложение хранит все моменты.

Две типичные задачи. Первая — *проверка однородности*: есть две выборки, и нужно понять, из одного ли они распределения; например, не изменились ли данные между обучением модели и её работой в проде. Статистикой критерия[7] служит выборочная MMD: если она заметно больше, чем получается при случайном перемешивании двух выборок, гипотеза об одном распределении отвергается. Порог получают перестановками: выборки объединяют, случайно перемешивают, снова делят на n

[7] Gretton A. et al. A Kernel Two-Sample Test. JMLR, 2012.

и m объектов и пересчитывают статистику; несколько сотен таких повторов дают её распределение при гипотезе $P = Q$. В отличие от классических критериев, которые либо сравнивают только средние, либо работают с одномерными данными, этот различает любые два распределения и применим ко всему, на чём задано ядро, — от многомерных векторов до графов; слабое место — зависимость чувствительности от выбора ядра. Вторая задача — *оценка генеративных моделей*: нужно сравнить распределение сгенерированных картинок с распределением настоящих. Сами картинки сравнивать бессмысленно, поэтому каждую пропускают через предобученную классифицирующую сеть и берут выход её предпоследнего слоя — вектор из нескольких тысяч чисел; сравнивают распределения этих векторов. Стандартная метрика FID считает, что оба распределения нормальны, и сравнивает их средние и ковариации. Kernel Inception Distance[8] — это MMD на тех же векторах с полиномиальным ядром третьей степени $(\langle x, z\rangle/d + 1)^3$: предположения о нормальности нет, а оценка по выборке несмещённая, так что её можно считать на малых выборках.

Нейронные сети как ядровые методы. Рассмотрим сеть $a(x;\theta)$ с параметрами θ и линеаризуем её по параметрам вокруг инициализации θ_0:

$$a(x;\theta) \approx a(x;\theta_0) + \langle \nabla_\theta a(x;\theta_0), \theta - \theta_0\rangle.$$

Это линейная модель над признаками $\varphi(x) = \nabla_\theta a(x;\theta_0)$: роль весов w играет вектор $\theta - \theta_0$, а признаками служат частные производные выхода сети по каждому параметру. Как и для любой линейной модели, обучение можно переписать через ядро $K(x,z) = \langle \varphi(x), \varphi(z)\rangle$; для него есть своё обозначение

$$\Theta(x,z) = \langle \nabla_\theta a(x;\theta_0), \nabla_\theta a(z;\theta_0)\rangle$$

и своё название — neural tangent kernel, касательное ядро сети: «касательное», потому что линеаризация — это касательная к a в точке θ_0.

Посмотрим, что делает с такой моделью градиентный спуск.

1. Обучаем линеаризованную модель по весам $w = \theta - \theta_0$ с квадратичной функцией потерь и без регуляризатора:

$$Q(w) = \frac{1}{2}\sum_{i=1}^{\ell}\big(a(x_i;\theta_0) + \langle \varphi(x_i), w\rangle - y_i\big)^2 = \frac{1}{2}\sum_{i=1}^{\ell}\big(\langle \varphi(x_i), w\rangle - r_i\big)^2 \to \min_w,$$

   где $r_i = y_i - a(x_i;\theta_0)$. Это линейная регрессия на остатки r_i; стартуем из $w_0 = 0$. Обозначим через Φ матрицу размера $\ell \times p$ со строками $\varphi(x_i)^T$ (p — число параметров), через r — вектор остатков; тогда $Q(w) = \frac{1}{2}\|\Phi w - r\|^2$, а матрица Грама ядра Θ на обучающей выборке равна $\Theta = \Phi\Phi^T$.

2. Шаг градиентного спуска:

$$w_{t+1} = w_t - \eta\nabla Q(w_t) = w_t - \eta\Phi^T(\Phi w_t - r) = w_t + \eta\sum_{i=1}^{\ell}\big(r_i - \langle \varphi(x_i), w_t\rangle\big)\varphi(x_i).$$

   Каждый шаг добавляет к w линейную комбинацию векторов $\varphi(x_i)$, а стартуем мы из нуля, поэтому $w_t = \Phi^T\alpha_t$ для некоторого $\alpha_t \in \mathbb{R}^\ell$.

[8] Bińkowski M. et al. Demystifying MMD GANs. ICLR, 2018.

3. Найдём w_t явно. Для остатков на обучающей выборке $e_t = \Phi w_t - r$ из формулы шага получаем

$$e_{t+1} = \Phi w_{t+1} - r = e_t - \eta\Phi\Phi^T e_t = (I - \eta\Theta)e_t, \qquad e_t = (I-\eta\Theta)^t e_0 = -(I-\eta\Theta)^t r.$$

Веса — сумма сделанных шагов:

$$w_t = -\eta\Phi^T(e_0 + e_1 + \cdots + e_{t-1}) = \eta\Phi^T \sum_{s=0}^{t-1}(I - \eta\Theta)^s\, r.$$

Это геометрическая прогрессия с матричным знаменателем $I - \eta\Theta$. Если Θ положительно определена и $0 < \eta < 2/\lambda_{\max}(\Theta)$, все собственные значения знаменателя по модулю меньше единицы, и сумма равна $(\eta\Theta)^{-1}\big(I - (I - \eta\Theta)^t\big)$. Отсюда

$$w_t = \Phi^T\Theta^{-1}\big(I - (I - \eta\Theta)^t\big)r.$$

Предсказание линеаризованной модели после t шагов обозначим $a_t^{\text{lin}}(x) = a(x;\theta_0) + \langle\varphi(x), w_t\rangle$; поскольку $\varphi(x)^T\Phi^T = k_\Theta(x)^T$, где $k_\Theta(x) = (\Theta(x, x_i))_{i=1}^{\ell}$,

$$a_t^{\text{lin}}(x) = a(x;\theta_0) + k_\Theta(x)^T\Theta^{-1}\big(I - (I - \eta\Theta)^t\big)r.$$

При $t \to \infty$ степень $(I - \eta\Theta)^t$ стремится к нулю, и

$$a_t^{\text{lin}}(x) \xrightarrow[t\to\infty]{} a_\infty^{\text{lin}}(x) = a(x;\theta_0) + k_\Theta(x)^T\Theta^{-1}r.$$

Это формула ядровой регрессии из раздела 1.2 при $\lambda = 0$, с ядром Θ, обученной на остатках r: к ядровой регрессии сходится последовательность функций a_t^{lin}. Видно и как именно: по собственным направлениям Θ с большими собственными значениями ошибка убывает быстро, с малыми — медленно, так что ранняя остановка работает как регуляризатор.

Всё выше — про линеаризованную модель, и верно оно для любой сети и любой точки θ_0; ядро Θ при этом зависит от θ_0. Вопрос в том, когда настоящая, нелинейная сеть ведёт себя так же. На него отвечает теорема об NTK[9], и суть её в следующем: достаточно широкая сеть, обученная градиентным спуском из случайной инициализации, — это ядровой метод, и его ядро можно выписать по архитектуре, не обучая сеть. Точнее (обозначим $\Theta(x, z;\theta) = \langle\nabla_\theta a(x;\theta), \nabla_\theta a(z;\theta)\rangle$ — ядро при произвольных параметрах; выше $\Theta(x, z) = \Theta(x, z;\theta_0)$):

- При стремлении ширины слоёв к бесконечности (и подходящей нормировке инициализации)

$$\Theta(x, z;\theta_0) \to \Theta_\infty(x, z) \text{ по вероятности при ширине} \to \infty,$$

где Θ_∞ — детерминированное ядро, зависящее только от глубины, функции активации и дисперсии инициализации. Оно не меняется и в ходе обучения: для градиентного потока на любом конечном отрезке времени $[0, T]$

$$\sup_{t\in[0,T]} \big|\Theta(x, z;\theta_t) - \Theta_\infty(x, z)\big| \to 0 \text{ по вероятности при ширине} \to \infty$$

(T фиксировано, ширина растёт). То есть ядро — характеристика архитектуры, как гауссово ядро характеризуется параметром σ.

[9] Jacot A., Gabriel F., Hongler C. Neural Tangent Kernel: Convergence and Generalization in Neural Networks. NeurIPS, 2018.

- Обозначим через $a_t(x)$ предсказание настоящей сети после t шагов того же градиентного спуска по θ. На всём протяжении обучения она не отличается от своей линеаризации[10]: для спуска с шагом $\eta < 2/(\lambda_{\min}(\Theta) + \lambda_{\max}(\Theta))$ при $\lambda_{\min}(\Theta) > 0$

$$\sup_t \left| a_t(x) - a_t^{\text{lin}}(x) \right| \to 0 \text{ при ширине} \to \infty,$$

  причём расхождение имеет порядок $n^{-1/2}$ по ширине n. Поэтому предсказания обученной сети — это $a_\infty^{\text{lin}}(x) = a(x;\theta_0) + k_\Theta(x)^T \Theta^{-1} r$ из пункта 3: чтобы узнать, что выучит сеть, достаточно решить линейную систему с матрицей Θ.

Одна деталь: $a(x;\theta_0)$ при случайной инициализации — случайная функция, и a_∞^{lin} раскладывается на ядровую регрессию на метки y и центрированную случайную добавку $a(x;\theta_0) - k_\Theta(x)^T \Theta^{-1} a_0$, равную нулю на обучающих объектах. Так что «совпадают с ядровой регрессией» верно с точностью до этой добавки; на практике её усредняют по инициализациям или вычитают $a(x;\theta_0)$ из выхода сети. Отсюда два следствия. Сходимость: если матрица Θ невырождена, градиентный спуск доводит ошибку на обучении до нуля — один из первых результатов такого рода для нейронных сетей. Обобщение: качество на новых данных такое же, как у ядровой регрессии с ядром Θ, и его можно изучать средствами теории ядровых методов.

Чего теорема не утверждает — что реальные сети так и работают. Содержательно предел бесконечной ширины означает, что параметры за всё обучение почти не сдвигаются, и градиенты $\nabla_\theta a(x;\theta_t)$ остаются такими же, как при инициализации: внутренние представления не меняются, обучаются только веса над случайными признаками $\nabla_\theta a(x;\theta_0)$ — как в Random Kitchen Sinks, о которых пойдёт речь на следующей лекции. Реальные сети конечной ширины ведут себя иначе: их представления после обучения сильно отличаются от начальных и переносятся на другие задачи (на этом стоит всё предобучение), а по качеству сеть обгоняет ядровую регрессию со своим же ядром Θ, и разрыв растёт с глубиной. Именно это отклонение от предела и даёт сетям преимущество перед ядровыми методами. NTK — инструмент для теории, а не описание глубинного обучения целиком.

Ядра и внимание. В трансформере для запроса q_i и ключей $k_1, \dots, k_n$ выход внимания равен

$$\frac{\sum_{j=1}^n \exp\big(\langle q_i, k_j\rangle/\sqrt{d}\big) v_j}{\sum_{j=1}^n \exp\big(\langle q_i, k_j\rangle/\sqrt{d}\big)},$$

то есть это ядровое сглаживание значений v_j с ядром $K(q,k) = \exp(\langle q,k\rangle/\sqrt{d})$ — а это ядро, как мы видели. Здесь d — размерность векторов q, k и v. Посчитаем операции. Для одного запроса q_i числитель требует n скалярных произведений длины d и n сложений векторов длины d:

$$\sum_{j=1}^n \underbrace{\exp\big(\langle q_i, k_j\rangle/\sqrt{d}\big)}_{O(d)} v_j \qquad \Rightarrow \qquad O(nd) \text{ на запрос}, \quad O(n^2 d) \text{ на все } n \text{ запросов},$$

[10] Lee J. et al. Wide Neural Networks of Any Depth Evolve as Linear Models under Gradient Descent. NeurIPS, 2019. У Jacot et al. результат доказан для градиентного потока на конечном отрезке времени; Lee et al. переносят его на дискретный спуск и все t.

и столько же, $n \times n$, занимает матрица весов внимания. Если найти признаки $\tilde{\varphi} \in \mathbb{R}^m$ с $\langle \tilde{\varphi}(q), \tilde{\varphi}(k) \rangle \approx K(q,k)$, то числитель перепишется так:

$$\sum_{j=1}^{n} \tilde{\varphi}(q_i)^T \tilde{\varphi}(k_j)\, v_j = \tilde{\varphi}(q_i)^T \underbrace{\sum_{j=1}^{n} \tilde{\varphi}(k_j)\, v_j^T}_{S \in \mathbb{R}^{m \times d},\ O(nmd)\ \text{один раз}} \quad \Rightarrow \quad \underbrace{\tilde{\varphi}(q_i)^T S}_{O(md)} \text{ на запрос.}$$

Матрица S общая для всех запросов и накапливается за один проход по ключам; знаменатель аналогично равен $\tilde{\varphi}(q_i)^T z$, где $z = \sum_j \tilde{\varphi}(k_j) \in \mathbb{R}^m$ считается за $O(nm)$. Добавим вычисление самих признаков $\tilde{\varphi}$ для всех q_i и k_j — ещё $O(nmd)$. Итого:

$$\underbrace{O(n^2 d)}_{\text{обычное внимание}} \quad \text{против} \quad \underbrace{O(nmd)}_{\text{признаки}} + \underbrace{O(nmd)}_{S} + \underbrace{O(nmd)}_{\text{запросы}} = O(nmd),$$

и вместо матрицы $n \times n$ хранится матрица $m \times d$. Экономия есть, когда число признаков m много меньше длины последовательности n. Для экспоненциального ядра есть точное представление через гауссовские случайные векторы: при $w \sim \mathcal{N}(0, I)$

$$\exp(\langle q, k \rangle) = \mathbb{E}_w \Big[\exp\big(\langle w, q \rangle - \|q\|^2/2\big) \exp\big(\langle w, k \rangle - \|k\|^2/2\big) \Big],$$

и $\tilde{\varphi}$ строится усреднением по m сэмплам w[11]; это близкий родственник случайных признаков Фурье, которые мы разберём на следующей лекции. В чистом виде линейное внимание стандартное не вытеснило: точное внимание научились считать быстро на уровне памяти GPU (FlashAttention), а качество у приближения ниже. Идея живёт в гибридных архитектурах, где слои линейного внимания чередуются с обычными, как правило три к одному: так устроены Qwen3-Next и Qwen3.5, Kimi Linear, Ling 2.5 и ряд других открытых моделей 2025–2026 годов.

Список литературы

[1] *Bishop, C.M.* Pattern Recognition and Machine Learning. // Springer, 2006.

[2] *Shawe-Taylor, J., Cristianini, N.* Kernel Methods for Pattern Analysis. // Cambridge University Press, 2004.

[3] *Schölkopf, B., Smola, A.J.* Learning with kernels. // MIT Press, 2002.

[4] *Micchelli, C.A.* Algebraic aspects of interpolation. // Proceedings of Symposia in Applied Mathematics, 36:81-102, 1986.

[11] Choromanski K. et al. Rethinking Attention with Performers. ICLR, 2021.